

Supervised learning.

Numbers and classes.

Predict a rating. Estimate a thumbs-up.

Learn what changes when the target changes.

🕒 18-22 min core reading · 20-30 min practice · 12 short sections

Prepared data becomes useful when a model learns a relationship between inputs and observed outcomes. Now we will fit two small models, inspect what they predict, and separate learning from the decisions built around those predictions.

WHAT YOU WILL UNDERSTAND

Two questions: “What rating might this viewer give?” and “Will this viewer give a thumbs-up?” Learn why they need different outputs and losses.

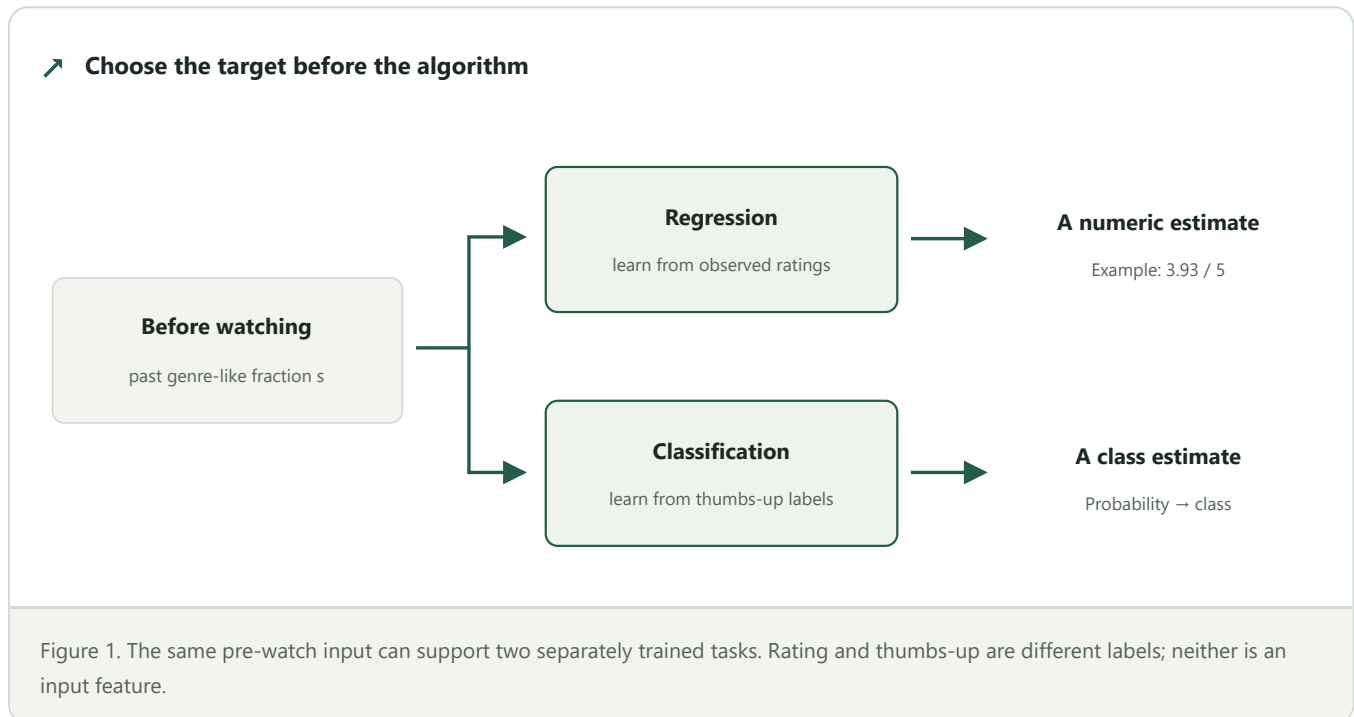
The practical path: fit a line, fit a probability curve, compare a baseline, and change a decision threshold without changing the model.

Reading path: concise core explanations, three diagrams, a working model explorer, and a downloadable Python workbook. Expand the deeper notes when needed.

1. Start with the outcome you want to predict

Supervised learning uses examples that pair inputs with known outcomes. For a historical movie-viewing event, the input describes what was known before watching; the outcome records later feedback. Training adjusts model parameters to reduce a chosen loss on those examples. Inference applies the fitted model to a new input whose outcome is not yet known.

Regression predicts a numerical quantity, such as a rating of 3.93. **Classification** predicts membership in a category, such as thumbs-up or thumbs-down. A classifier may first estimate class probabilities and then apply a decision rule.



The target's meaning determines the task. A movie ID such as 104 is an identifier, not a quantity to predict with regression merely because it is written as a number. Likewise, a probability between 0 and 1 can be an intermediate output of a classification model.

▼ Go deeper: binary, multiclass, multilabel, and ordinal targets

Binary classification has two classes, such as liked/not liked. **Multiclass classification** chooses one class from several, such as one primary genre. **Multilabel classification** allows several labels at once: a film could be both adventure and comedy. These tasks require different target representations and output rules.

A one-to-five star rating is **ordinal**: the order is meaningful, but equal gaps between adjacent ratings are an assumption. Our regression exercise treats rating differences as numerical distances. That is a modeling choice, not a fact about human taste; an ordinal model or a class-based formulation could suit another product. Neither approach makes a predicted rating an observed fact.

2. Give each model a clear input and label

Chapter 12 introduced the preparation boundary. We keep it: separate viewers, fit only on training examples, and exclude later feedback from features. This chapter uses a **new synthetic snapshot** with a history feature and two labels; it does not add invented ratings to Chapter 12's original records.

The input s is the fraction of a viewer's **earlier rated movies in this genre** that received a thumbs-up. For example, three likes among four earlier rated adventure films gives $s = 0.75$. This is observed history, not the model's probability for the next film.

Training event	Past fraction s	Later rating	Later thumbs-up
S01	0.0	1	0
S02	0.2	2	0
S03	0.4	3	1
S04	0.6	3	0
S05	0.8	4	1
S06	1.0	5	1

Each event belongs to a different viewer. Three additional viewers supply validation events and three supply test events. Rating and thumbs-up are separately recorded feedback; the thumbs-up label is not calculated by thresholding the rating. The two three-star events deliberately have different thumbs-up labels.

We transform s into $x = 2s - 1$, putting the input between -1 and 1 . This fixed formula uses no estimated dataset statistics. At $s = 0.75$, $x = 0.5$. Both models receive only this single feature, not the labels or viewer IDs.

▼ Go deeper: historical features need a time boundary

In a real system, compute the fraction from feedback available before the target viewing event. Including that event's own outcome-or later feedback-leaks the answer. A viewer can be absent from model training yet bring legitimate earlier history at prediction time. That is the deployment situation this grouped example represents; it is not a no-history cold-start experiment.

The workbook supplies authored fractions, not raw timestamped histories, so it cannot verify their provenance. It also omits history counts: $3/4$ and $300/400$ both produce 0.75 but have different uncertainty. With zero earlier rated movies, the fraction is undefined; do not silently set it to zero. A production design needs a missing-history policy, suitable features, and evaluation for that population.

3. Fit a line for a numerical prediction

Our **linear regression** model is:

$$\hat{y} = wx + b.$$

The hat means “predicted.” The weight w controls how the prediction changes with x ; the intercept b is the prediction at $x = 0$. These are learned **parameters**. With several features, the model becomes $\hat{y} = w_1x_1 + \dots + w_dx_d + b$, connecting directly to Chapter 6’s dot product.

Without regularization, fitting our six rows gives $w = 13/7 \approx 1.8571$ and $b = 3$. For a viewer with $s = 0.75$, the prediction is:

$$\hat{y} = \frac{13}{7}(0.5) + 3 \approx 3.9286.$$

The line summarizes a tendency rather than passing through every point. It predicts the same value for identical inputs, even when different viewers later react differently. One history feature cannot describe every reason a person likes a film.

▼ Go deeper: where the fitted numbers come from

For one nonconstant feature, least squares has a direct solution: divide the sum of centered input-target products by the sum of squared centered inputs, then recover the intercept. Here the input mean is 0, the rating mean is 3, the product sum is 5.2, and the squared-input sum is 2.8. Therefore $w = 5.2 / 2.8 = 13/7$ and $b = 3$. This is a **closed-form solution**; the program does not need a gradient-descent loop to fit this line.

Other solvers can optimize the same objective. “Linear” describes how the learned coefficients enter the formula; transformed features can still represent curved relationships. Our unconstrained line does not guarantee outputs within 1-5 for arbitrary inputs or datasets. Clipping changes the delivered prediction and must be evaluated as part of the system. A fitted association also does not prove that changing someone’s history would cause a rating to change. See scikit-learn’s [linear-model guide](#).

4. Measure regression errors in the right units

Define the prediction error as $e_i = \hat{y}_i - y_i$. If a model predicts 4 but the observed rating is 3, the error is +1. A signed average could let positive and negative errors cancel, so common losses use absolute or squared errors.

Measure	Calculation	Interpretation
MAE	Average of absolute errors	Typical absolute error in rating points
MSE	Average of squared errors	Rating points squared; larger errors matter more
RMSE	Square root of MSE	Error scale in rating points, sensitive to larger errors

For predictions $[4, 2]$ and observations $[3, 4]$, errors are $[1, -2]$. MAE is $(1 + 2) / 2 = 1.5$; MSE is $(1 + 4) / 2 = 2.5$; RMSE is about 1.5811. The units help you avoid treating MSE 2.5 as “off by 2.5 stars.” See [Google’s regression-loss explanation](#).

A **baseline** gives an improvement something to beat. Our regression baseline always predicts the training mean rating, 3. On the three validation events, its MSE is 2.6667; our unregularized line gets about 0.1878. That comparison describes these authored examples, not a real-world accuracy claim.

5. Fit a probability curve for a class

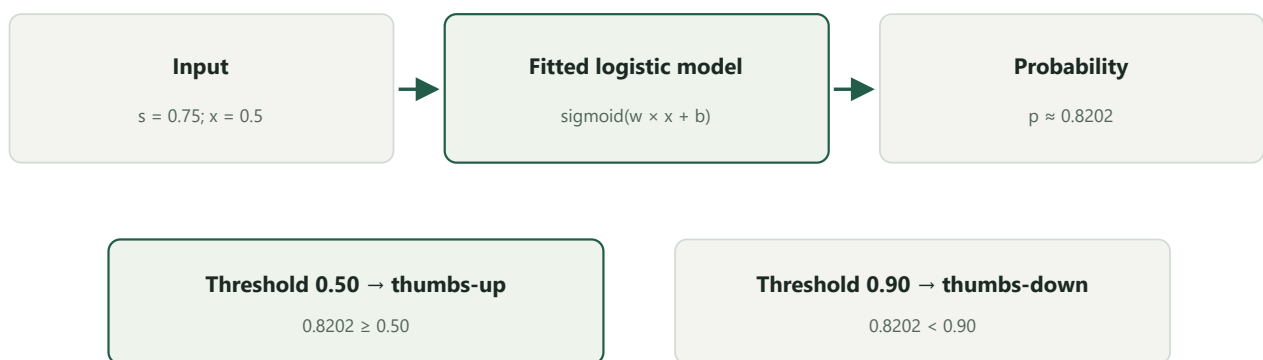
Logistic regression, despite its name, is commonly used for classification. It first computes a real-valued score, or **logit**, $z = wx + b$, then converts it through the **sigmoid** function:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The output estimates $P(\text{thumbs-up} = 1 \mid x)$. A score of 0 maps to 0.5; a score of 1 maps to about 0.7311; negative scores map below 0.5. The sigmoid bounds the mathematical output between 0 and 1. It does not automatically establish accurate probabilities. See [Google’s sigmoid walkthrough](#).

Our fitted default classifier has $w \approx 3.0351$ and $b \approx 0$. For $s = 0.75$, $x = 0.5$, so $z \approx 1.5175$ and $p \approx 0.8202$. This probability concerns a thumbs-up; it is neither an 82% rating nor a guarantee about this viewer.

➤ A probability and a decision are different outputs



Same trained weights. Same probability. Different decision policy.

Figure 2. For the fitted default classifier at history $s = 0.75$, p is about 0.8202. Changing the threshold changes the class while the fitted model stays fixed.

▼ Go deeper: calibration and more than two classes

A probability estimate is **calibrated** when events assigned probabilities near 0.8 occur about 80% of the time over an appropriate set of cases. A sigmoid alone does not establish this; calibration must be checked on representative held-out data. Six synthetic training events are insufficient evidence. Distribution changes can also make previously useful probabilities unreliable.

For one-of-many classes, a model can produce one score per class and use **softmax** to turn them into probabilities summing to 1. For multilabel problems, separate sigmoid outputs can represent labels that may coexist; they need not sum to 1. The target meaning determines the appropriate output, loss, and decision policy. These models will connect to neural-network output layers later.

6. Train classification with a probability-sensitive loss

Binary cross-entropy, also called binary log loss, compares an estimated probability p with a label y that is 0 or 1:

$$\ell(y, p) = -[y \ln p + (1 - y) \ln(1 - p)].$$

When $y = 1$, the loss is $-\ln p$. Assigning the observed thumbs-up probability 0.8 costs about 0.2231; assigning it 0.2 costs about 1.6094. Confident mistakes receive a large penalty. With $y = 0$, the relevant term is $-\ln(1 - p)$. We average over training events.

Accuracy counts correct class decisions. It ignores whether a correct decision used probability 0.51 or 0.99. Log loss distinguishes those predictions, so two classifiers can have equal accuracy and different log losses. These are different questions, not interchangeable scores.

The workbook starts $w = b = 0$ and takes 2,000 full-batch gradient steps with learning rate 0.3. Each step averages prediction-minus-label errors to update the parameters. This is an actual fit on the six training rows, not an authored probability table. The fixed step count is a teaching budget, not a general convergence guarantee. The loss connects to Chapter 8's cross-entropy and gradient descent; see [Google's logistic-loss guide](#).

▼ Go deeper: the update you can inspect in Python

For average binary cross-entropy, the derivative with respect to each logit is proportional to $p - y$. The workbook computes $dw = \text{mean}((p - y) \times x) + \lambda w$ and $db = \text{mean}(p - y)$, then updates w and b by subtracting 0.3 times their gradients. λ is the L2 strength explained below; the intercept is not penalized. Both gradients use the same old parameters before either update is applied.

The code evaluates log loss from logits with a stable equivalent expression, $\max(z, 0) - yz + \log(1 + \exp(-|z|))$. This avoids computing $\log(0)$ after a probability rounds to 0 or 1. Finite-difference checks verify that the implemented gradient matches the objective. Numerical care preserves the calculation; it does not improve the data's representativeness.

7. Choose a decision threshold separately

Our policy predicts thumbs-up when $p \geq t$, where t is a threshold. At the default $t = 0.5$, probability 0.8202 becomes thumbs-up. Raising the threshold to 0.9 changes it to thumbs-down. The weights and probability stay unchanged.

The **decision boundary** is where the class changes. At threshold 0.5, the logistic model's boundary is $wx + b = 0$. With our nearly zero intercept, this is near $x = 0$, or $s = 0.5$. In multiple input dimensions, a basic logistic model has a linear boundary at a fixed threshold; other models can learn nonlinear boundaries.

A **false positive** predicts thumbs-up for an observed thumbs-down. A **false negative** misses an observed thumbs-up. Raising a threshold reduces the set predicted positive; it can avoid some unwanted recommendations while missing appealing ones. Choose that tradeoff using validation data and the product's costs, not the final test set. See scikit-learn's [threshold guide](#).

8. Control flexibility and know the model families

Underfitting means the model misses relevant structure, even in training. **Overfitting** means it fits training-specific details that do not carry over well. A small training-validation gap alone does not prove a useful model; both sets can perform poorly.

Regularization discourages certain fitted solutions. Our L2 penalty is $\lambda w^2/2$. Larger λ pushes the weight toward zero, making the response flatter. The regression objective is half the mean squared error plus this penalty; the classification objective is mean log loss plus the penalty. These scaling conventions matter when comparing implementations.

The weight and intercept are learned parameters. Strength λ , learning rate, and chosen model family are **hyperparameters**: settings chosen around training, commonly with validation. Excessive regularization can underfit; it never guarantees a better score.

➤ A tree makes local decisions

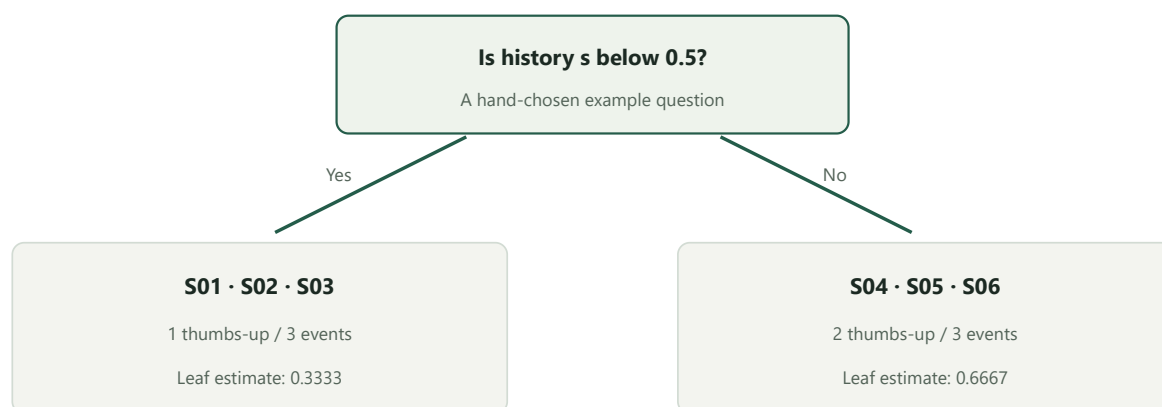


Figure 3. An illustrative, hand-chosen split of our six training events. Each leaf reports its observed thumbs-up fraction. The workbench fits linear and logistic models, not this tree.

A **decision tree** learns feature-based questions and predictions for the resulting regions. In classification, splits can reduce label mixing, measured by quantities such as Gini impurity or entropy. Regression trees can reduce squared-error variation and predict leaf averages. Limiting depth or leaf size controls how finely the tree divides the data. See [scikit-learn's tree guide](#).

▼ Go deeper: a compact map of other supervised methods

k-nearest neighbors predicts from nearby stored examples: averaging their targets for regression or voting for classification. Its neighborhood size and distance definition matter. A **support vector machine** seeks a boundary with a large margin; kernels can represent nonlinear relationships. Scaling can materially affect both methods.

Random forests combine randomized trees to reduce instability. **Gradient boosting** adds models sequentially to improve the ensemble's objective. **Naive Bayes** combines class and feature probabilities under a conditional-independence assumption. **Neural networks** learn composed transformations and are developed later in this handbook. These families offer different assumptions, flexibility, costs, and interpretability; none wins on every dataset. This chapter's executable models remain linear and logistic regression. See the primary [supervised-learning index](#) for the individual methods.

9. Explore two fitted models

Begin with regression, no penalty, and history fraction 0.75. Confirm the predicted rating of about 3.9286. Switch to validation events and compare the model with the constant baseline; displaying another split does not refit the model.

Change to classification, then move the threshold from 0.50 to 0.90. The decision changes while the displayed weight and probability remain fixed. Set L2 strength to 1: now the model is refitted, and the probability becomes about 0.5261 because its response is flatter.

One input. Two learning tasks.

Use a viewer's past enjoyment of this genre to predict a rating or a thumbs-up. Six training events fit the model. Three validation events let you inspect its behavior.

Default query: $s = 0.75$, $x = 0.5$; $\lambda = 0$ and classification threshold = 0.5. Models fit only the six training rows.

Task	Prediction	Training loss	Validation loss
regression	3.9286	0.0571	0.1878
classification	0.8202	0.4130	0.3418

Regression losses are MSE in rating points². Classification losses are average negative log likelihood (natural logs). Their scales cannot be directly compared.

The rating is 3.9286; estimated thumbs-up probability is 0.8202. Threshold 0.50 predicts thumbs-up; threshold 0.90 predicts thumbs-down with the same probability and weights. At $\lambda = 1$ the rating becomes 3.2955 and the probability becomes 0.5261, illustrating shrinkage toward a constant baseline.

Validation regression MSE: model 0.1878 versus baseline 2.6667. Classification: both get 2 of 3 validation labels correct at threshold 0.5, but log losses differ: 0.3418 versus 0.6931. These are synthetic arithmetic results, not evidence of real-world quality.

The rating and thumbs-up tasks use different labels and losses. Comparing regression MSE directly with classification log loss cannot tell you which task is "better." In the default classification comparison, the model and baseline each get two of three validation events correct, yet their log losses differ. Chapter 15 will develop evaluation and model selection more fully.

10. Run the workbook and practice

Extract the four files into a folder and open a terminal there. Python 3.10 or newer is sufficient; no third-party packages or API calls are needed.

```
py supervised.py
py supervised.py --task classification
py supervised.py --task classification --threshold 0.9
py supervised.py --task classification --strength 1
py -m unittest -v test_supervised.py
```

Use your working `python` or `python3` command if `py` is unavailable. The script reads `movie_learning.json` beside itself and prints fitted parameters, a query prediction, and training/validation metrics. It leaves input files unchanged. The browser independently implements the same calculations in JavaScript; parity checks compare both versions.

After fixing your chosen configuration, append `--evaluate-test` to report its test metrics. The flag is a workflow reminder, not a lock against repeated peeking. Three synthetic test events still cannot establish real-world usefulness. Invalid data or out-of-range query values stop with a clear error and exit code 2.

Exercise 1: follow the rating calculation

For a viewer whose history fraction is 0.75, calculate x , then use $w = 13/7$ and $b = 3$ to predict a rating. What is the constant baseline's prediction?

▼ Reveal the reasoning

$x = 2 \times 0.75 - 1 = 0.5$. The model gives $(13/7) \times 0.5 + 3 \approx 3.9286$. The baseline always gives 3, the mean training rating. Neither result tells us the viewer's actual future feedback.

Exercise 2: separate learning from policy

With classification and no regularization, inspect history 0.75. Compare thresholds 0.5 and 0.9. Then hold the threshold fixed and change regularization to 1. Which operation changes the fitted weight?

▼ Reveal the reasoning

The threshold edit keeps $w \approx 3.0351$ and $p \approx 0.8202$ while changing the class. Changing L2 strength refits the model: w becomes about 0.2090 and p about 0.5261. In the browser, threshold and query edits reuse the fitted model; a new CLI run recomputes the same fit unless its training choices change.

Exercise 3: verify the training boundary

In a copy of the dataset, change validation event S08's rating from 3 to 5. Rerun default regression. Should the fitted line or its validation MSE change?

▼ Reveal the reasoning

The fitted weight, intercept, and training predictions stay unchanged because S08 is not a training row. Validation MSE changes because it compares predictions with the edited target. The Python tests verify that held-out edits cannot change fitted parameters. Restore the original snapshot before comparing with the handbook's numbers.

11. Check your understanding

01 You want to estimate how many rating points a viewer will give. Which task matches that goal?

- A. Binary classification only.
- B. Regression, if rating differences are treated as numeric quantities.
- C. Clustering without labels.

Answer: B. Regression predicts a numerical target. Treating star ratings as numeric makes an assumption about the meaning of differences between ratings.

02 The classifier keeps probability 0.82 while the threshold changes from 0.5 to 0.9. What changes?

- A. The predicted class, but not the learned weights or probability.
- B. The training labels.
- C. The model is necessarily retrained.

Answer: A. With $p \geq \text{threshold}$, 0.82 passes 0.5 but fails 0.9. This is a decision-policy change around the same fitted model.

03 Why use log loss alongside accuracy?

- A. Log loss proves that probabilities are calibrated.
- B. It is always the same number as accuracy.
- C. It distinguishes probability estimates even when class decisions match.

Answer: C. Assigning probability 0.99 to the wrong class receives a larger penalty than a less confident error. Accuracy only counts the final class decisions.

04 What does larger L2 strength do in this workbook?

- A. Guarantee a better validation score.
- B. Penalize large weights more strongly, often making the fitted response flatter.
- C. Add extra viewer information to the dataset.

Answer: B. Regularization trades fit against coefficient size. Excessive shrinkage can underfit; validation must guide the choice.

05 The model beats a baseline on three synthetic validation events. What follows?

- A. It is ready to recommend movies to everyone.
- B. It proves that genre history causes enjoyment.
- C. It beat that baseline on those examples; broader performance is still unestablished.

Answer: C. A tiny authored dataset is useful for learning the mechanics, not for demonstrating generalization, calibration, or causality.

12. Connect the pieces before choosing a bigger model

You can now trace **available inputs** → **fitted parameters** → **numeric prediction or probability** → **evaluation** → **application decision**. Keep each boundary explicit. A predicted high rating does not override Chapter 11's hard runtime constraint, and a thumbs-up probability does not establish streaming availability.

For a new task, define the target, establish a baseline, fit on training data, and compare suitable losses on held-out examples. Change model capacity and settings for a reason. More flexible algorithms cannot recover missing context merely by being more complex.

▼ Quick-reference glossary

Term	Meaning here
Supervised learning	Learn input-outcome relationships from labeled examples
Regression / classification	Predict a numerical quantity / a class
Parameter	Value learned during fitting, such as weight or intercept
Hyperparameter	Setting chosen around fitting, such as L2 strength
Loss	Numerical penalty for predictions compared with labels
Baseline	Simple reference prediction to compare against
Logit / sigmoid	Unbounded linear score / mapping of that score to 0-1
Log loss	Probability-sensitive negative log likelihood
Threshold	Cutoff used to turn a probability into a class
Decision boundary	Input locations where the predicted class changes
Calibration	Agreement between predicted probabilities and observed frequencies
Regularization	A constraint or penalty that discourages some fitted solutions

Primary references: Google's [regression losses](#) and [logistic predictions](#); scikit-learn's [linear models](#), [decision trees](#), and [decision thresholds](#). The examples, diagrams, and numerical results here use our original synthetic movie data.

Next: Chapter 14 - Unsupervised learning and other learning approaches. Explore structure without the same kind of labeled target, and compare clustering, dimensionality reduction, and alternative learning signals.